

Les Parties du Code, Agent par Agent

Les 8 parties habituelles d'un script d'agent IA, appliquees au vrai code des projets 38 a 42.

38. SiteMap-Firecrawl

38-SiteMap-Firecrawl/map_agent.py

1. Imports

```
from firecrawl import Firecrawl
from pydantic import BaseModel, Field
from agno.agent import Agent
from agno.models.groq import Groq
```

Pourquoi

Rien de special ici par rapport aux autres agents : Firecrawl pour la donnee, Pydantic pour la structurer, Agno+Groq pour le LLM.

2. Configuration

```
FIRECRAWL_API_KEY = os.getenv("FIRECRAWL_API_KEY")
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
MAP_LIMIT = 40
```

Pourquoi

MAP_LIMIT=40 est plus haut que dans les autres agents (SEARCH_LIMIT=3 a 5 ailleurs) : map() ne lit aucun contenu, donc on peut se permettre de demander beaucoup plus de liens sans risquer de depasser un quota de tokens.

3. Client externe

```
firecrawl = Firecrawl(api_key=FIRECRAWL_API_KEY)
```

Pourquoi

Une seule instance, reutilisee par explore_site().

4. Modeles de donnees

```
class Link(BaseModel):
    url: str
    title: str

class Category(BaseModel):
    name: str = Field(description="...")
    urls: list[str] = Field(description="...")

class SiteMapResult(BaseModel):
    categories: list[Category]
```

Pourquoi

3 modeles imbriques : Link (une entree brute), Category (un groupe de liens), SiteMapResult (la liste complete de categories). Le Field(description=...) est envoye au LLM comme partie du schema qu'on lui demande de remplir.

5. Definition de l'agent

```
categorizer_agent = Agent(
    model=Groq(api_key=GROQ_API_KEY, id="openai/gpt-oss-120b"),
    name="Site Map Categorizer",
    role="Expert en architecture de site web",
    instructions=dedent("""...ne pas inventer d'URL..."""),
    output_schema=SiteMapResult,
)
```

Pourquoi

output_schema=SiteMapResult force une sortie JSON validee plutot que du texte libre -- indispensable puisque l'interface affiche les categories dans une grille de cartes, pas comme un paragraphe.

6. Fonction de recuperation

```
def explore_site(url, search=None, limit=MAP_LIMIT) -> list[Link]:
    result = firecrawl.map(url, search=search, limit=limit)
    ...
    return links
```

Pourquoi

Appelle firecrawl.map() et filtre les liens sans URL valide.

7. Fonction d'orchestration

```
def map_site(url, search=None, limit=MAP_LIMIT):
    links = explore_site(url, search=search, limit=limit)
    categories = categorize(links)
    return links, categories
```

Pourquoi

Explore puis categorise -- 2 etapes, 1 seul appel LLM (contrairement a l'agent 39 qui a aussi 2 etapes mais avec un contenu bien plus lourd a traiter).

8. Interface (extrait)

```
with st.form("map_form"):
    url = st.text_input(...)
    submitted = st.form_submit_button("Cartographier le site")

if submitted and url.strip():
    links, categories = map_site(url.strip(), ...)
```

Pourquoi

Le formulaire capture l'URL, le bouton declenche map_site(). Le reste du fichier (majoritaire) est du CSS pour le design -- pas de logique metier.

39. SiteCrawl-Firecrawl

39-SiteCrawl-Firecrawl/crawl_agent.py

1. Imports

```
from firecrawl import Firecrawl
from firecrawl.v2.types import ScrapeOptions
from pydantic import BaseModel
from agno.agent import Agent
from agno.models.groq import Groq
```

Pourquoi

ScrapeOptions en plus par rapport à l'agent 38 : nécessaire ici car crawl() doit aussi lire le contenu de chaque page (pas juste lister les URLs).

2. Configuration

```
CRAWL_LIMIT = 5
MAX_CHARS_PER_PAGE = 2500
```

Pourquoi

CRAWL_LIMIT=5 est volontairement bas (contre MAP_LIMIT=40 dans l'agent 38) : crawl() scrape réellement chaque page, l'appel Firecrawl le plus coûteux en crédits.

3. Client externe

```
firecrawl = Firecrawl(api_key=FIRECRAWL_API_KEY)
```

Pourquoi

Identique au pattern des autres agents.

4. Modeles de donnees

```
class Page(BaseModel):
    title: str
    url: str
    content: str
```

Pourquoi

Un seul modèle ici (pas de sortie structurée pour le rapport final : digest_agent écrit du markdown libre, pas du JSON, contrairement à l'agent 38).

5. Definition de l'agent

```
digest_agent = Agent(
    model=Groq(api_key=GROQ_API_KEY, id="openai/gpt-oss-120b"),
    name="Site Digest Writer",
    instructions=dedent("""## Vue d'ensemble ... ## Themes principaux ..."""),
    markdown=True,
)
```

Pourquoi

markdown=True (pas output_schema) : la sortie est un texte structuré en sections Markdown, affiche tel quel -- pas besoin de JSON puisque rien n'est réaffiché sous forme de tableau/grille ici.

6. Fonction de recuperation

```
def crawl_site(url, limit=CRAWL_LIMIT) -> list[Page]:
    job = firecrawl.crawl(url, limit=limit, scrape_options=ScrapeOptions(formats=["markdown"]))
    ...
    return pages
```

Pourquoi

La plus défensive des fonctions de récupération (getattr partout) car crawl() touche plusieurs pages, chacune pouvant échouer indépendamment.

7. Fonction d'orchestration

```
def digest(url, limit=CRAWL_LIMIT):
    pages = crawl_site(url, limit=limit)
    if not pages:
        return "Aucune page exploitable...", []
    ...
    report = digest_agent.run(prompt)
    return report.content, pages
```

Pourquoi

Le garde-fou 'if not pages' evite un appel LLM inutile si le crawl n'a rien trouve -- meme logique defensive que dans les 4 autres agents.

8. Interface (extrait)

```
limit = st.slider("Nombre de pages a explorer", min_value=2, max_value=10, value=CRAWL_LIMIT)
...
report, pages = digest(url.strip(), limit=limit)
```

Pourquoi

Le slider expose CRAWL_LIMIT a l'utilisateur -- lui permet d'arbitrer lui-meme entre richesse du rapport et cout en credits.

40. PageAnalyzer-Firecrawl

40-PageAnalyzer-Firecrawl/scrape_agent.py

1. Imports

```
from firecrawl import Firecrawl
from agno.agent import Agent
from agno.models.groq import Groq
```

Pourquoi

Le plus court des 5 : pas de Pydantic ici, car aucune sortie structuree n'est necessaire (juste une reponse texte).

2. Configuration

```
MAX_CONTENT_CHARS = 12000
```

Pourquoi

La plus haute limite de caracteres des 5 agents (12000, contre 900-2500 ailleurs) : puisqu'une seule page est traitee a la fois (pas de cumul de plusieurs sources dans le meme prompt), on peut se permettre d'en garder beaucoup plus.

3. Client externe

```
firecrawl = Firecrawl(api_key=FIRECRAWL_API_KEY)
```

Pourquoi

Identique aux autres.

4. Modeles de donnees

Pourquoi

Seul agent des 5 sans classe Pydantic : la sortie est une reponse en texte libre a une question, pas une donnee a re-afficher sous forme structuree.

5. Definition de l'agent

```
page_agent = Agent(
    model=Groq(api_key=GROQ_API_KEY, id="openai/gpt-oss-120b"),
    name="Page Analyzer",
    instructions=dedent("""Si une question est posee : repondez-y... Sinon : resume..."""),
    markdown=True,
)
```

Pourquoi

Les instructions couvrent explicitement 2 scenarios (avec/sans question) -- c'est le seul agent des 5 concu pour avoir 2 comportements differents selon l'entree.

6. Fonction de recuperation

```
def scrape_page(url) -> Optional[str]:
    result = firecrawl.scrape(url, formats=["markdown"])
    markdown = getattr(result, "markdown", None)
    if not markdown or not markdown.strip():
        return None
    return markdown.strip()
```

Pourquoi

La plus simple des 5 : une seule page, pas de boucle.

7. Fonction d'orchestration

```
def analyze_page(url, question=None):
    content = scrape_page(url)
    if content is None:
        return None, "Impossible d'extraire..."
    if question and question.strip():
        prompt = f"Question : ...{content}"
    else:
        prompt = f"Resume cette page : ...{content}"
    answer = page_agent.run(prompt)
    return answer.content, None
```

Pourquoi

C'est ICI (pas dans l'agent) que le choix entre question-reponse et resume est fait : deux prompts differents construits selon la presence de `question`.

8. Interface (extrait)

```
url = st.text_input("URL de la page :", ...)
question = st.text_input("Question (optionnel) :", ...)
...
answer, error = analyze_page(url.strip(), question=question.strip() or None)
```

Pourquoi

Deux champs (URL + question optionnelle) au lieu d'un seul -- reflète directement le double comportement de la fonction d'orchestration.

41. DocumentCompare-MistralOCR

41-DocumentCompare-MistralOCR/compare_agent.py

1. Imports

```
from typing import Literal
from mistralai.client import Mistral
from pydantic import BaseModel, Field
from agno.agent import Agent
from agno.models.mistral import MistralChat
```

Pourquoi

Premier agent des 5 a utiliser Mistral plutot que Firecrawl+Groq : mistralai pour l'OCR ET le LLM, agno.models.mistral pour brancher Mistral sur Agno. `Literal` est nouveau ici (restreint le champ `type` a 3 valeurs exactes).

2. Configuration

```
MAX_CHARS_PER_DOC = 15000
```

Pourquoi

Contexte Mistral plus large que Groq, donc limite plus genereuse par document que dans les agents Firecrawl.

3. Client externe

```
mistral_client = Mistral(api_key=MISTRAL_API_KEY)
```

Pourquoi

Un seul client Mistral sert 2 usages differents : l'OCR brut (ocr_pdf) ET, via MistralChat, le LLM de comparaison -- les 2 autres agents utilisant un seul provider partagent ce pattern.

4. Modeles de donnees

```
class Change(BaseModel):
    type: Literal["Ajout", "Suppression", "Modification"] = Field(...)
    clause: str = Field(...)
    detail: str = Field(...)

class ComparisonResult(BaseModel):
    summary: str = Field(...)
    changes: list[Change] = Field(...)
```

Pourquoi

Le `Literal` garantit que `type` est TOUJOURS l'une des 3 valeurs exactes -- important car l'interface utilise cette valeur comme nom de classe CSS pour les badges colores (chg-badge.Ajout, etc.) : une faute d'orthographe du LLM casserait l'affichage sans ce garde-fou.

5. Definition de l'agent

```
comparison_agent = Agent(
    model=MistralChat(api_key=MISTRAL_API_KEY, id="mistral-large-latest"),
    name="Document Comparator",
    instructions=dedent("""...Ne signalez que des changements de FOND..."""),
    output_schema=ComparisonResult,
)
```

Pourquoi

mistral-large-latest (plus puissant que les modeles Groq utilises ailleurs) : une tache de comparaison fine entre deux textes demande plus de rigueur qu'une simple synthese.

6. Fonction de recuperation

```
def ocr_pdf(file_path) -> str:
    uploaded = mistral_client.files.upload(...)
    signed_url = mistral_client.files.get_signed_url(file_id=uploaded.id)
    ocr_response = mistral_client.ocr.process(model="mistral-ocr-latest", ...)
    return "\n".join(page.markdown for page in ocr_response.pages)
```

Pourquoi

Seule fonction de recuperation des 5 qui lit un FICHIER (PDF) plutot qu'une URL web -- 3 etapes (upload, URL signee, OCR) cachees derriere une signature simple (chemin -> texte).

7. Fonction d'orchestration

```
def compare_pdfs(path_a, path_b):  
    text_a = ocr_pdf(path_a)  
    text_b = ocr_pdf(path_b)  
    result = compare_documents(text_a, text_b)  
    return result, text_a, text_b
```

Pourquoi

Seule orchestration des 5 qui appelle la fonction de recuperation 2 FOIS (un par document) avant l'etape de decision.

8. Interface (extrait)

```
col_a, col_b = st.columns(2)  
with col_a:  
    file_a = st.file_uploader("Document A", type=["pdf"], key="file_a")  
with col_b:  
    file_b = st.file_uploader("Document B", type=["pdf"], key="file_b")
```

Pourquoi

Deux zones d'upload cote a cote (pas un champ texte) : seul agent des 5 dont l'entree est un FICHIER a televerser plutot qu'une URL a saisir.

42. ResearchTeam-Agno

42-ResearchTeam-Agno/research_team.py

1. Imports

```
from firecrawl import Firecrawl
from firecrawl.v2.types import ScrapeOptions
from agno.agent import Agent
from agno.team.team import Team
from agno.models.groq import Groq
```

Pourquoi

Seul agent des 5 a importer `Team` (agno.team.team) : les 4 autres utilisent un `Agent` seul, celui-ci en coordonne plusieurs.

2. Configuration

```
def groq_model(model_id=MODEL_ID) -> Groq:
    return Groq(id=model_id, api_key=GRQ_API_KEY, retries=2, delay_between_retries=2, exponential_backoff=True)
```

Pourquoi

Seul agent des 5 avec une fonction-fabrique pour le modele (retries integres) : necessaire car un test reel a declenche 96 depassements de quota Groq sur une seule execution d'equipe -- les autres agents (mono-appel) n'ont jamais eu ce probleme.

3. Client externe

```
firecrawl = Firecrawl(api_key=FIRECRAWL_API_KEY)
```

Pourquoi

Meme pattern que les agents Firecrawl, mais utilise ici a l'interieur d'un OUTIL plutot que d'une fonction de recuperation classique.

4. Modeles de donnees

Pourquoi

Comme l'agent 40 : les 2 membres de l'equipe repondent en texte libre (markdown=True), pas de schema JSON.

5. Definition des agents (x2) + de l'equipe

```
searcher_agent = Agent(model=groq_model(), name="Chercheur", tools=[firecrawl_search], ...)
writer_agent = Agent(model=groq_model(), name="Redacteur", ...)

research_team = Team(
    members=[searcher_agent, writer_agent],
    model=groq_model(),
    mode="coordinate",
    delegate_to_all_members=True,
    instructions=dedent("""1. Deleguez au Chercheur... 2. Deleguez au Redacteur..."""),
)
```

Pourquoi

3 "agents" ici : 2 membres + 1 leader (le Team lui-meme a son propre modele). Le leader ne fait rien seul : il orchestre les 2 membres selon les instructions -- une orchestration faite par un LLM plutot que par du code Python pur.

6. Fonction de recuperation (comme OUTIL)

```
def firecrawl_search(query: str) -> str:
    """...Utilisez cet outil pour trouver de l'information..."""
    result = firecrawl.search(query, limit=SEARCH_LIMIT, ...)
    ...
    return "\n\n".join(blocks)
```

Pourquoi

Difference fondamentale avec les 4 autres agents : cette fonction n'est jamais appelee directement dans le code. Elle est donnee via tools=[firecrawl_search] au Chercheur, qui decide LUI-MEME quand l'appeler. Le docstring sert de mode d'emploi lu par le LLM, pas juste de documentation pour un humain.

7. Fonction d'orchestration

```
def run_research(query):
    try:
        result = research_team.run(query)
    except Exception as e:
        return None, f"Erreur : {e}"
    content = result.content
    if content.strip().startswith('{ "error" }'):
        return None, "Le fournisseur du modele a refuse..."
    return content, None
```

Pourquoi

La plus defensive des 5 : verifie non seulement les exceptions, mais aussi un echec SILENCIEUX (une erreur JSON deguisee en reponse valide) -- decouvert lors d'un vrai test rate, pas anticipe a l'avance.

8. Interface (extrait)

```
st.markdown(
    '<div class="team-chip">Chercheur</div>'
    '<div class="team-arrow">-&gt;</div>'
    '<div class="team-chip">Redacteur</div>'
)
...
report, error = run_research(query.strip())
```

Pourquoi

Seule interface des 5 a visualiser explicitement le PIPELINE (chips + fleche) plutot qu'un simple formulaire -- reflete qu'il y a plusieurs etapes/agents impliquees, pas un seul appel direct.